

Sistema Experto de Recomendaciones Academicas

Documento de Diseno y Arquitectura

Implementado en Common Lisp (SBCL)

Bachillerato en Ingenieria en Sistemas de Computacion

Universidad Fidelitas

Motor de inferencia propio · 25 reglas declarativas

47 cursos · 358 pruebas automatizadas

Agosto de 2026

Contenido

1. Proposito y alcance del sistema
2. Arquitectura general
 1. Division en capas
 2. Invariante de dependencias
 3. Estructura de archivos
3. Flujo de una sesion completa
4. El motor de inferencia
 1. Ciclo match → select → act
 2. Resolucion de conflictos
 3. Refraccion y terminacion
 4. La traza
5. Modelo de conocimiento
 1. Las tres capas de hechos
 2. Catalogo de relaciones
 3. Reglas de modelado
6. Reglas implementadas (25)
7. Reglas de negocio y su trazabilidad
8. Ejemplos implementados
 1. Los cinco perfiles
 2. Salida real del sistema
 3. Explicacion reconstruida
 4. Traza del motor
9. Estadisticas producidas
10. Verificacion y criterios de aceptacion
11. Limitaciones y procedencia de los datos

1. Proposito y alcance del sistema

El sistema recomienda cursos universitarios a un estudiante a partir de sus intereses, sus cursos aprobados, su horario disponible, su tolerancia a la dificultad y su area profesional objetivo, y **explica** cada recomendacion enumerando las reglas que la produjeron. Ademas entrega estadisticas del estudiante y del catalogo.

El valor del proyecto no esta en ordenar una lista de cursos: eso se resolveria con un filtro y un ordenamiento en unas pocas lineas. Esta en que un **motor de inferencia propio**, con reglas declarativas y trazabilidad completa, produce y justifica la recomendacion. Esa distincion gobierna todas las decisiones de diseno que siguen.

Criterio de diseno dominante. El conocimiento academico vive fuera del programa: son datos que el motor interpreta. Agregar conocimiento nuevo significa agregar una regla, nunca modificar el motor. Una prueba automatizada verifica exactamente eso.

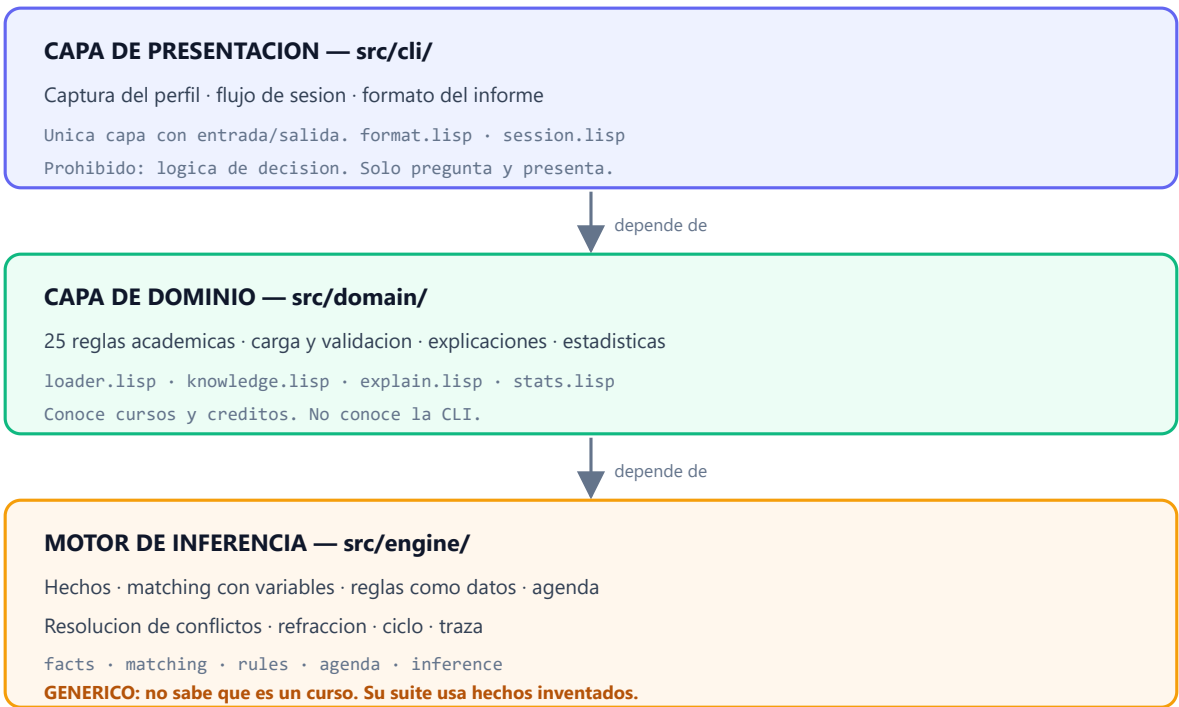
Alcance

Incluido	Excluido deliberadamente
Motor de encadenamiento hacia adelante escrito desde cero	Librerias de reglas (LISA, CLIPS)
Base de conocimiento auditable en texto	Aprendizaje automatico
Recomendaciones con explicacion trazable	Interfaz web o grafica
Estadisticas de estudiante y de catalogo	Matricula real o conexion a la universidad
Interfaz de linea de comandos interactiva	Horarios con hora exacta y traslapes parciales

2. Arquitectura general

2.1 Division en capas

El sistema se divide en tres capas con dependencias en una sola direccion. Cada una tiene una responsabilidad que las otras no pueden asumir.



2.2 Invariante de dependencias

Las flechas apuntan siempre hacia abajo. La regla que sostiene todo el diseno es la del motor:

El motor no puede saber que es un curso. Ningun archivo de `src/engine/` menciona un simbolo del dominio academico. Su suite de pruebas opera exclusivamente con hechos inventados como `(color rojo)`. Si el motor necesitara conocer el dominio para pasar sus pruebas, dejaria de ser un motor y seria un programa de recomendacion disfrazado.

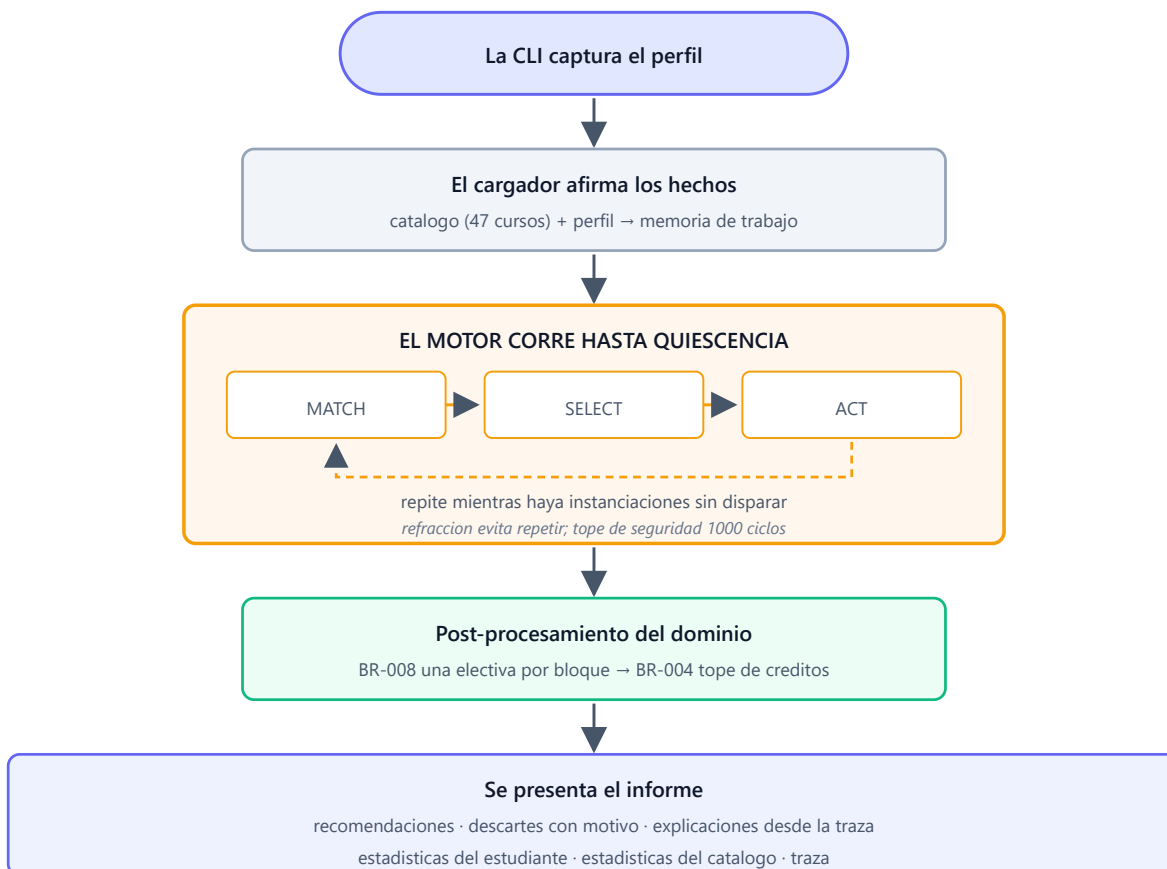
Capa	Puede usar	Tiene prohibido
cli/	El dominio	Decidir nada; solo pregunta y presenta
domain/	El motor	Entrada/salida; <code>format t</code> fuera de la CLI
engine/	Solo Common Lisp	Cualquier simbolo del dominio academico

2.3 Estructura de archivos

```
src/  
  package.lisp      Los tres paquetes y sus exportaciones  
  engine/  
    facts.lisp      Memoria de trabajo, hechos con id monotono  
    matching.lisp    Unificacion de patrones con variables  
    rules.lisp       Macro DEFRULE y registro de reglas  
    agenda.lisp      Conjunto de conflicto, resolucion, refraccion  
    inference.lisp    Ciclo match-select-act, quiescencia, traza  
  domain/  
    loader.lisp      Carga y validacion de data/  
    knowledge.lisp    Las 25 reglas + post-procesamiento  
    explain.lisp      Explicaciones reconstruidas desde la traza  
    stats.lisp        Estadisticas de estudiante y de catalogo  
  cli/  
    format.lisp       Todo el texto que ve el usuario  
    session.lisp      Captura interactiva y flujo de sesion  
  main.lisp          Punto de entrada  
  
data/  
  courses.lisp       Catalogo de 47 cursos (datos, no codigo)  
  profiles/  
  
tests/              Espejo de src/ + suite de aceptacion
```

3. Flujo de una sesion completa

Desde que el estudiante responde la primera pregunta hasta que ve el informe, el recorrido es el siguiente. Notese que el motor corre una sola vez, hasta agotar todo lo que puede concluir.



Por que hay post-procesamiento despues del motor. Dos reglas — el tope de creditos (BR-004) y el limite de una electiva por bloque (BR-008) — comparan entre si un conjunto de tamano variable. El matching por patrones posicionales tiene aridad fija y no expresa esa clase de agregacion, asi que se implementan como funciones de dominio que corren tras la quiescencia. Es una limitacion conocida del modelo de matching, documentada, no un atajo.

4. El motor de inferencia

Es un motor de produccion con **encadenamiento hacia adelante**: parte de los datos y deriva conclusiones, en vez de partir de una meta e intentar probarla. Se eligio asi porque la tarea real es "dame todos los cursos que me convienen", que en encadenamiento hacia atras obligaria a probar la meta una vez por cada curso del catalogo.

4.1 Ciclo match → select → act

Fase	Que hace
MATCH	Evalua las condiciones de todas las reglas contra la memoria de trabajo y construye el conjunto de conflicto: todas las instanciaciones aplicables.
SELECT	Ordena el conjunto de conflicto y elige una sola instanciacion, descartando las ya disparadas.
ACT	Afirma los hechos de la parte <code>:then</code> con las variables sustituidas, y registra el disparo en la traza.

4.2 Resolucion de conflictos

Cuando varias reglas pueden dispararse, el motor elige de forma **determinista**, en este orden:

#	Criterio	Justificacion
1	Mayor prioridad de la regla	Permite ordenar por franjas: una franja completa se agota antes de que empiece la siguiente
2	Hecho mas reciente	Medido por el id de insercion monotono de la memoria de trabajo
3	Orden de declaracion	Desempate final: garantiza que la misma entrada produzca siempre la misma traza

Por que las prioridades van de 5 en 5. Cuando una regla niega un hecho que *otra regla* deriva, esa otra debe haber disparado ya en todos los cursos, o la negacion daria un falso positivo en un ciclo temprano. Las franjas 40, 35, 30, 25, 20, 15 y 10 garantizan ese agotamiento por niveles.

4.3 Refraccion y terminacion

Una misma instanciacion — la misma regla con las mismas ligaduras — no vuelve a dispararse. Sin refraccion, una regla que afirma un hecho que vuelve a satisfacer su propia condicion haria que el motor no terminara nunca. Existe ademas un tope de seguridad de 1000 ciclos que senala una advertencia si se alcanza; la demostracion completa usa alrededor de 295.

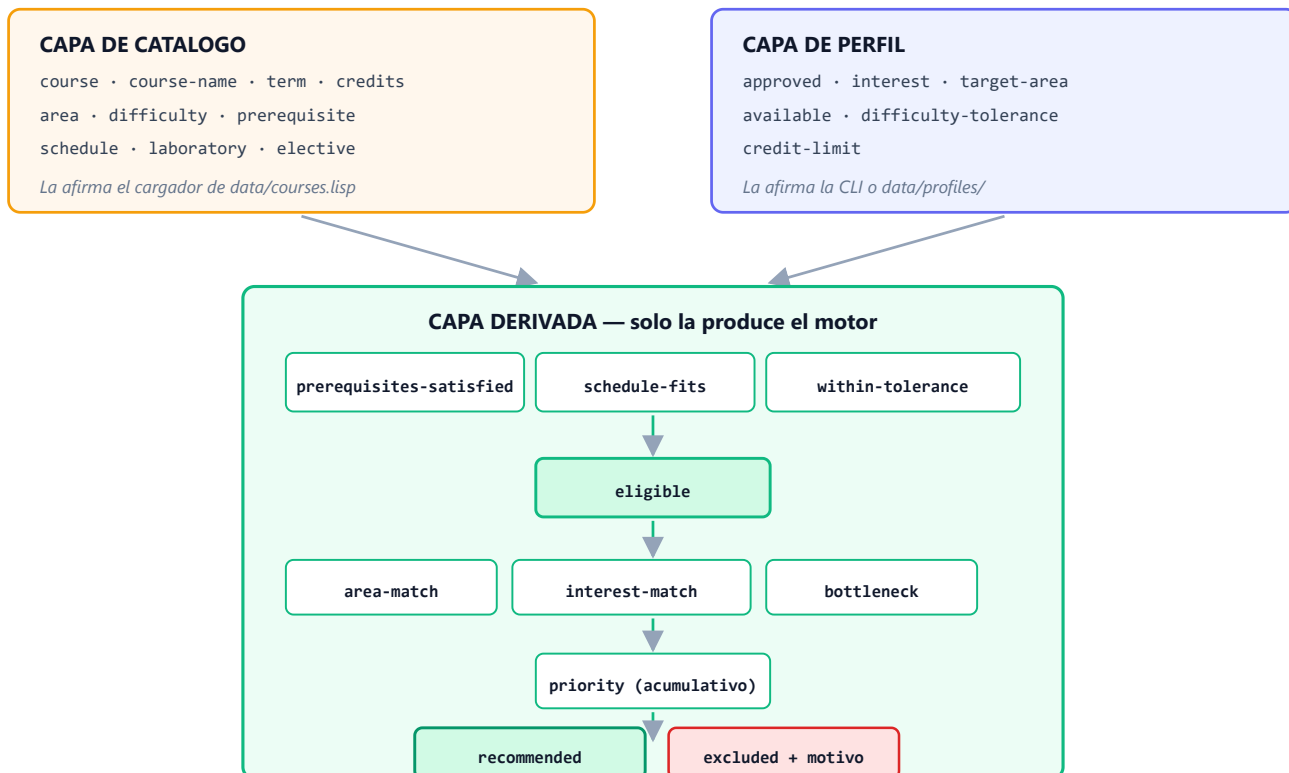
4.4 La traza

Cada disparo registra la regla, las ligaduras, los hechos que la activaron y los hechos que produjo. **La explicacion al estudiante no se escribe a mano: se reconstruye recorriendo esta traza** hacia atras desde el hecho `(recommended ?id)`. Por eso la explicacion refleja el razonamiento real y no una narracion paralela que podria desincronizarse.

5. Modelo de conocimiento

5.1 Las tres capas de hechos

Todo lo que el motor conoce es un hecho: una lista plana cuyo primer elemento nombra la relacion. No hay marca que diga si un hecho es dato de entrada o conclusion; la distincion esta en *que relacion se usa* y *quien puede afirmarla*. Esa disciplina es lo que hace legible la traza.



Invariante. Ninguna regla puede afirmar una relacion de la capa de catalogo o de la capa de perfil. Si una regla lo necesitara, el modelo de datos estaria mal planteado y habria que revisarlo antes de escribir codigo.

5.2 Catalogo de relaciones

Relacion	Capa	Significado
(course id)	Catalogo	Existencia de un curso
(course-name id nombre)	Catalogo	Nombre legible; ninguna regla condiciona sobre el
(term id n)	Catalogo	Cuatrimestre del plan (1–8)
(credits id n)	Catalogo	Creditos del curso
(area id area)	Catalogo	Area profesional
(difficulty id n)	Catalogo	Dificultad en escala 1–5
(prerequisite id req)	Catalogo	Un hecho por requisito; nunca listas anidadas
(schedule id dia franja)	Catalogo	Un hecho por bloque de horario
(elective id grupo)	Catalogo	Bloque electivo al que pertenece
(approved id)	Perfil	Curso ya ganado por el estudiante
(interest area)	Perfil	Area que le atrae; puede haber varias
(target-area area)	Perfil	Area profesional objetivo; exactamente una
(available dia franja)	Perfil	Bloque en que puede llevar clases
(difficulty-tolerance n)	Perfil	Umbral de dificultad declarado
(credit-limit n)	Perfil	Tope de creditos del semestre
(eligible id)	Derivada	Puede llevarlo: requisitos, horario y no aprobado
(bottleneck id n)	Derivada	Es requisito de n cursos
(priority id n)	Derivada	Puntaje acumulativo: varias reglas suman
(recommended id)	Derivada	Se le presenta al estudiante
(excluded id razon)	Derivada	Descartado, con el motivo explicito

5.3 Reglas de modelado

#	Regla	Por que
1	Aridad fija	Una relacion tiene siempre el mismo numero de argumentos
2	Sin anidamiento	Ningun argumento es una lista; se repite el hecho. Mantiene el matcher simple
3	Identificadores string, categorias simbolo	Los codigos se comparan con <code>equal</code> ; las categorias con <code>eq</code>
4	Sin negacion en los hechos	No existe <code>(not-approved id)</code> ; la ausencia se expresa en las condiciones
5	Una relacion, un significado	Un matiz nuevo es una relacion nueva, no un argumento extra

6. Reglas implementadas

Las **25 reglas** que siguen se extrajeron del sistema en ejecucion, no se transcribieron a mano: el documento se genera leyendo las estructuras que el motor tiene cargadas, de modo que no puede desfasarse del codigo. Cada una se declara con la macro `defrule` y es una estructura de datos inspeccionable, no codigo ejecutable.

```
(engine:defrule nombre-de-la-regla
  "Docstring que cita el BR que implementa."
  :priority 30
  :when ((course ?id) (not (approved ?id)))
  :then ((eligible ?id)))
```

Elegibilidad (BR-001 a BR-003)

missing-prerequisite-detected

prioridad 40

Auxiliar de BR-001: ?id tiene al menos un requisito sin aprobar.

```
:when ((prerequisite ?id ?req) (not (approved ?req)))
:then ((missing-prerequisite ?id))
```

prerequisites-satisfied-when-none-missing

prioridad 35

Implementa BR-001: todos los requisitos de ?id estan aprobados.

```
:when ((course ?id) (not (missing-prerequisite ?id)))
:then ((prerequisites-satisfied ?id))
```

schedule-block-unavailable-detected

prioridad 40

Auxiliar de BR-003: ?id tiene un bloque de horario fuera de la disponibilidad declarada por el estudiante.

```
:when ((schedule ?id ?day ?slot) (not (available ?day ?slot)))
:then ((schedule-block-unavailable ?id))
```

schedule-fits-when-no-conflict

prioridad 35

Implementa BR-003: todos los bloques de horario de ?id caen dentro de la disponibilidad declarada.

```
:when ((course ?id) (not (schedule-block-unavailable ?id)))
:then ((schedule-fits ?id))
```

eligible-when-prerequisites-and-schedule-ok

prioridad 30

Implementa BR-001, BR-002 y BR-003: ?id es elegible si no esta aprobado, sus requisitos estan satisfechos y su horario calza.

```
:when ((course ?id) (not (approved ?id)) (prerequisites-satisfied ?id) (schedule-fits ?id))
:then ((eligible ?id))
```

excluded-missing-prerequisites

prioridad 10

Implementa BR-021 para BR-001.

```
:when ((course ?id) (not (approved ?id)) (missing-prerequisite ?id))
:then ((excluded ?id missing-prerequisites))
```

Cuello de botella (BR-006)

bottleneck-with-three-dependents

prioridad 25

Implementa BR-006: ?id es prerrequisito de 3 o mas cursos distintos. La prueba PRECEDES exige un orden estricto entre los tres dependientes para que dispare una sola vez por umbral alcanzado, en vez de una vez por cada permutacion de los mismos tres cursos.

```
:when ((prerequisite ?c1 ?id) (prerequisite ?c2 ?id) (prerequisite ?c3 ?id) (engine:precedes ?c1 ?c2)
(engine:precedes ?c2 ?c3))
:then ((bottleneck ?id +bottleneck-threshold+))
```

Tolerancia a la dificultad (BR-005)

within-tolerance-rule

prioridad 25

Implementa BR-005: la dificultad de ?id no excede la tolerancia declarada por el estudiante.

```
:when ((course ?id) (difficulty ?id ?d) (difficulty-tolerance ?t) (engine:at-most ?d ?t))
:then ((within-tolerance ?id))
```

bottleneck-exception-to-tolerance

prioridad 25

Implementa la excepcion de BR-005: un cuello de botella que excede la tolerancia por exactamente un nivel se recomienda igual, marcado con advertencia.

```
:when ((course ?id) (difficulty ?id ?d) (difficulty-tolerance ?t) (bottleneck ?id ?n) (engine:exceeds-by-one ?d ?t))
:then ((within-tolerance ?id) (tolerance-warning ?id))
```

Afinidad (soporte de BR-010, BR-011, BR-013)

area-match-rule

prioridad 25

?id coincide con el area profesional objetivo del estudiante.

```
:when ((area ?id ?a) (target-area ?a))
:then ((area-match ?id))
```

interest-match-rule

prioridad 25

?id coincide con alguno de los intereses declarados por el estudiante.

```
:when ((area ?id ?a) (interest ?a))
:then ((interest-match ?id))
```

Priorizacion (BR-010 a BR-015)

priority-target-area

prioridad 20

Implementa BR-010.

```
:when ((eligible ?id) (area-match ?id))
:then ((priority ?id +weight-target-area+))
```

priority-interest

prioridad 20

Implementa BR-011.

```
:when ((eligible ?id) (interest-match ?id))
:then ((priority ?id +weight-interest+))
```

priority-bottleneck

prioridad 20

Implementa BR-012.

```
:when ((eligible ?id) (bottleneck ?id ?n))
:then ((priority ?id +weight-bottleneck+))
```

priority-unlocks-target-area

prioridad 20

Implementa BR-013: ?id es requisito directo de un curso del area profesional objetivo.

```
:when ((eligible ?id) (prerequisite ?unlocked ?id) (area ?unlocked ?a) (target-area ?a))
:then ((priority ?id +weight-unlocks-target-area+))
```

priority-too-easy

prioridad 20

Implementa BR-014: la dificultad esta al menos 2 niveles por debajo de la tolerancia declarada.

```
:when ((eligible ?id) (difficulty ?id ?d) (difficulty-tolerance ?t) (engine:at-least-below ?t ?d 2))
:then ((priority ?id +weight-too-easy+))
```

priority-general-education

prioridad 20

Implementa BR-015. Simplificacion de esta primera version: se premia todo curso elegible de formacion general, sin contar cuantos le faltan al estudiante -- ese conteo es una agregacion que el matching por patrones posicionales no expresa (ver docs/adr/ADR-005 y la nota de limitaciones del proyecto).

```
:when ((eligible ?id) (area ?id general-education))
:then ((priority ?id +weight-general-education+))
```

De elegible a recomendado (BR-007)

recommended-via-target-area

prioridad 15

Implementa BR-007 via BR-010.

```
:when ((eligible ?id) (within-tolerance ?id) (area-match ?id))
:then ((recommended ?id))
```

recommended-via-interest

prioridad 15

Implementa BR-007 via BR-011.

```
:when ((eligible ?id) (within-tolerance ?id) (interest-match ?id))
:then ((recommended ?id))
```

recommended-via-bottleneck

prioridad 15

Implementa BR-007 via BR-012.

```
:when ((eligible ?id) (within-tolerance ?id) (bottleneck ?id ?n))
:then ((recommended ?id))
```

recommended-via-unlock

prioridad 15

Implementa BR-007 via BR-013.

```
:when ((eligible ?id) (within-tolerance ?id) (prerequisite ?unlocked ?id) (area ?unlocked ?a) (target-area ?a))
:then ((recommended ?id))
```

recommended-via-general-education

prioridad 15

Implementa BR-007 via BR-015.

```
:when ((eligible ?id) (within-tolerance ?id) (area ?id general-education))
:then ((recommended ?id))
```

Motivos de descarte (BR-021)

excluded-already-approved

prioridad 10

Implementa BR-002 + BR-021: no se recomienda lo ya aprobado.

```
:when ((course ?id) (approved ?id))
:then ((excluded ?id already-approved))
```

excluded-schedule-conflict

prioridad 10

Implementa BR-021 para BR-003.

```
:when ((course ?id) (not (approved ?id)) (prerequisites-satisfied ?id) (schedule-block-unavailable ?id))  
:then ((excluded ?id schedule-conflict))
```

excluded-too-difficult

prioridad 10

Implementa BR-021 para BR-005.

```
:when ((eligible ?id) (not (within-tolerance ?id)))  
:then ((excluded ?id too-difficult))
```

7. Reglas de negocio y su trazabilidad

El conocimiento experto se especifica primero en lenguaje natural, con un identificador BR, y despues se implementa. Cada `defrule` cita en su docstring el BR que implementa, de modo que la especificacion y el codigo se pueden confrontar en cualquier momento.

BR	Categoria	Descripcion
BR-001	Dura	Un curso es elegible solo si <i>todos</i> sus requisitos estan aprobados
BR-002	Dura	Nunca se recomienda un curso ya aprobado
BR-003	Dura	Todos los bloques de horario del curso deben caber en la disponibilidad declarada
BR-004	Dura	La suma de creditos recomendados no excede el tope del estudiante
BR-005	Estandar	No se recomienda un curso que excede la tolerancia a la dificultad
BR-006	Estandar	Un curso es cuello de botella si es requisito de 3 o mas cursos
BR-007	Estandar	De elegible a recomendado hace falta prioridad acumulada positiva
BR-008	Dura	De un mismo bloque electivo se conserva solo el de mayor prioridad
BR-010	Blanda	El area del curso es el area objetivo: +10
BR-011	Blanda	El area esta entre los intereses declarados: +5
BR-012	Blanda	El curso es cuello de botella: +8
BR-013	Blanda	Desbloquea un curso del area objetivo: +4
BR-014	Blanda	Muy por debajo de la tolerancia: -2
BR-015	Blanda	Curso de formacion general: +3
BR-020	Dura	Toda recomendacion debe tener explicacion no vacia
BR-021	Estandar	Toda exclusion se acompaña de su motivo
BR-030	Dura	Las estadísticas se derivan de la memoria de trabajo, nunca de una consulta aparte
BR-031	Estandar	Avance de carrera: creditos aprobados sobre creditos totales del catalogo

La excepcion de BR-005. Un curso cuello de botella que excede la tolerancia *por un solo nivel* se recomienda igual, marcado con advertencia: atrasarlo cuesta mas que llevarlo. Es el caso mas interesante del sistema porque muestra conocimiento experto real — una excepcion con criterio, no un filtro rigido.

8. Ejemplos implementados

8.1 Los cinco perfiles

Cada perfil de `data/profiles/` existe para ejercitar un comportamiento distinto del sistema, no para llenar. En conjunto activan las seis razones de descarte y las reglas de excepcion.

Perfil	Que representa	Que demuestra
<code>sample-profile</code>	Estudiante de segundo cuatrimestre	Las seis razones de descarte en una sola corrida
<code>first-year</code>	Primer ingreso, sin cursos aprobados	Solo recomienda cursos sin requisitos; avance de carrera en 0 %
<code>advanced</code>	Estudiante avanzado	Avance alto y activacion de las reglas de formacion general
<code>tight-schedule</code>	Disponibilidad muy restringida	El horario como filtro dominante (BR-003)
<code>low-tolerance</code>	Tolerancia a la dificultad baja	La excepcion de BR-005 : cuello de botella recomendado con advertencia

8.2 Salida real del sistema

Lo que sigue es salida literal de `data/profiles/low-tolerance.lisp`, sin editar. Es el caso mas ilustrativo: el sistema recomienda un curso que *excede* la tolerancia declarada y explica por que lo hace igual.

```
RECOMENDACIONES ACADEMICAS
```

```
PERFIL ANALIZADO
```

- Cursos aprobados: SC-202, SC-315, SC-115
- Intereses: mathematics
- Horarios disponibles: friday afternoon, friday morning, thursday afternoon, thursday morning, wednesday afternoon, wednesday morning, tuesday afternoon, tuesday morning, monday afternoon, monday morning
- Tolerancia de dificultad: 3
- Area profesional objetivo: software-engineering

```
RECOMENDACIONES
```

1. Estructura de Datos

8.3 Explicacion reconstruida

Las lineas de "Razones" y "Advertencias" no son texto fijo: cada una corresponde a una regla que efectivamente disparo sobre ese curso, recuperada de la traza. El puntaje 22 es la suma de los aportes de BR-012 (+8), BR-010 (+10) y BR-013 (+4), y cada sumando se puede rastrear hasta la regla que lo produjo.

Los descartes tambien se explican, con su motivo del vocabulario cerrado:

CURSOS DESCARTADOS

- Introduccion al Calculo o Matematica Basica
Motivo: No cabe dentro de tu tope de creditos, dado el orden de prioridad.
- Calculo Diferencial e Integral I
Motivo: No cabe dentro de tu tope de creditos, dado el orden de prioridad.
- Algebra Lineal
Motivo: No cabe dentro de tu tope de creditos, dado el orden de prioridad.
- Probabilidad y Estadistica Descriptiva

8.4 Traza del motor

El sistema puede mostrar el ciclo de inferencia completo, disparo por disparo. Esta es la evidencia de que existe un motor real y no logica cableada:

TRAZA DEL MOTOR

- Regla aplicada: schedule-block-unavailable-detected (ciclo 1)
Hecho generado: (schedule-block-unavailable SC-805)
- Regla aplicada: schedule-block-unavailable-detected (ciclo 2)
Hecho generado: (schedule-block-unavailable SC-270)
- Regla aplicada: schedule-block-unavailable-detected (ciclo 3)
Hecho generado: (schedule-block-unavailable SC-707)
- Regla aplicada: schedule-block-unavailable-detected (ciclo 4)
Hecho generado: (schedule-block-unavailable SC-704)
- Regla aplicada: schedule-block-unavailable-detected (ciclo 5)
Hecho generado: (schedule-block-unavailable SC-701)

9. Estadísticas producidas

Se calculan dos familias, ambas **derivadas de la memoria de trabajo final** (BR-030). No se consulta el catalogo por aparte, de modo que las cifras nunca pueden contradecir lo que el motor concluyo.

Del estudiante	Del catalogo y la base de conocimiento
Avance de carrera (BR-031)	Cursos mas recomendados entre perfiles
Creditos aprobados frente al total	Cursos cuello de botella, con cuantos desbloquean
Distribucion de aprobados por area	Dificultad promedio por area
Cursos evaluados, elegibles y descartados por motivo	Cobertura de reglas
Carga estimada del semestre propuesto	

La cobertura de reglas es una herramienta de diagnostico, no un adorno. Indica cuales de las 25 reglas nunca dispararon en una corrida. Una regla que jamas se activa es, o bien conocimiento muerto que sobra, o bien senal de que los datos no la ejercitan. En ambos casos es algo que hay que mirar.

Cobertura de reglas: 15 de 25 dispararon

Nunca dispararon (conocimiento muerto o datos que no lo ejercitan):

- bottleneck-exception-to-tolerance
- excluded-missing-prerequisites
- excluded-schedule-conflict
- excluded-too-difficult
- missing-prerequisite-detected

10. Verificacion y criterios de aceptacion

El sistema se verifica con **358 comprobaciones automatizadas**. El gate de verificacion compila el sistema completo con ASDf en orden de dependencias y corre la suite; termina imprimiendo un veredicto legible por maquina.

```
$ sh .ace/scripts/verify.sh
[ok] Gate 'test' passed.
All configured gates passed.
VERIFY_RESULT=pass gate=all

$ sbcl --script run-tests.lisp
Did 358 checks.
  Pass: 358 (100%)
  Skip: 0 ( 0%)
  Fail: 0 ( 0%)
```

Los diez criterios de aceptacion del sistema se verifican en una suite dedicada:

#	Criterio
1	Ninguna recomendacion viola un requisito, en ningun perfil
2	Ninguna recomendacion choca con el horario declarado
3	Toda recomendacion tiene explicacion no vacia (BR-020)
4	El motor es generico: su suite no usa ningun simbolo del dominio
5	Se puede agregar una regla en tiempo de ejecucion sin tocar el motor
6	El motor siempre termina, incluso con una regla que se reactiva a si misma
7	Misma entrada, misma salida: traza identica en corridas repetidas
8	El catalogo tiene entre 40 y 60 cursos
9	Ambas familias de estadisticas estan presentes
10	Una sesion completa corre en menos de dos segundos (medido: 0,3 s)

Los criterios 4 y 5 son los que sostienen la tesis del proyecto. El 4 demuestra que el motor es independiente del dominio; el 5, que el conocimiento se agrega como datos. Si alguno fallara, el sistema habria dejado de ser un sistema experto aunque siguiera dando buenas recomendaciones.

11. Limitaciones y procedencia de los datos

Esta seccion existe porque presentar datos provisionales como si fueran oficiales invalidaria el trabajo. El programa academico suministrado por la universidad aporta solo una parte de lo que el motor necesita para razonar.

Campo	Origen	Detalle
Codigo, nombre, cuatrimestre	Oficial	Tal como lo entrego la universidad
Laboratorio, curso colegiado	Oficial	Indicadores del programa
Bloque electivo	Oficial	El programa organiza las electivas en tres bloques
Creditos	Provisional	Valor uniforme de 4; el programa no los declara
Dificultad	Provisional	Formula: cuatrimestre, mas 1 si tiene laboratorio, tope 5
Area profesional	Provisional	Inferida por el equipo del nombre del curso
Horario	Provisional	Un bloque por curso, asignado por rotacion
Prerrequisitos	Provisional	Seis enlaces de demostracion; el programa no declara ninguno

Consecuencia que debe declararse al presentar el sistema. Los cuellos de botella que el sistema reporta son *correctos dado el grafo de requisitos cargado*, y ese grafo es de ejemplo. El razonamiento del motor es real y verificable; los datos de entrada son parciales. Si aparecieran los prerrequisitos oficiales, incorporarlos es un cambio en `data/` y **cero lineas de codigo**: la arquitectura mantiene los datos fuera del programa precisamente para eso.

Otras limitaciones conocidas

Limitacion	Alcance
Matching sin red RETE	El conjunto de conflicto se reconstruye entero en cada ciclo. Mitigado con un indice de hechos por relacion: sin el, cada condicion intentaba unificar contra los ~680 hechos de la sesion en vez de las decenas de su relacion, y una sesion pasaba de 0,3 s a mas de 3 s
Horarios por bloques discretos	No modela traslapes parciales ni horas exactas
Solo Bachillerato	El catalogo no cubre las licenciaturas
Agregaciones fuera del motor	BR-004 y BR-008 corren como post-procesamiento
Traza extensa	Cerca de 295 lineas en una corrida completa; no hay vista resumida

Documento generado a partir del codigo fuente del sistema. Las 25 reglas de la seccion 6 se extrajeron del motor en ejecucion; las salidas de la seccion 8 son literales, sin editar. Repositorio: github.com/Wardaddy118/Expert_System_Lisp